

CPU Execution Time in COA

Program $\longrightarrow$ Instructions $\longrightarrow$ Micro Operation / Instruction.

$\downarrow$
CPI
[Clock per Instruction].

Cycles $\rightarrow$ f_{clock} .

$\rightarrow$ Execution time, $t = \frac{n \times \text{CPI}_{\text{avg}}}{f_{\text{clock}}}$.

Average CPI in COA

Instruction Category	Numbers of Instructions	CPI
ALU	40	2
Load & Store	15	3
Branch	35	4
Other	10	5

$$\begin{aligned}CPI_{avg} &= 40 \times 2 + 15 \times 3 + 35 \times 4 \\ &\quad + 10 \times 5 \\ &\quad \hline &\quad 100 \\ &= 3.15\end{aligned}$$

$$CPI_{avg} = \frac{\sum_{i=1}^x n_i CPI_i}{\sum_{i=1}^x n_i}$$

where, n_i = no of instructions with i type

CPI_i = CPI of i type instruction

x = no of type instruction.

MIPS in COA

→ MIPS - Million Instructions per second.

$$\begin{aligned}\rightarrow \text{MIPS} &= \frac{n}{t \times 10^6} \\ &= \frac{\cancel{n}}{\cancel{n} \text{ CPI}_{\text{avg}} \times 10^6} \times f_{\text{clock}}\end{aligned}$$

$$\Rightarrow \boxed{\text{MIPS} = \frac{f_{\text{clock}}}{\text{CPI}_{\text{avg}} \times 10^6}}$$

Examples on CPU Performance in COA

- ❑ **GATE CS** – Find CPI_{avg} and MIPS for given specifications. {Clock = 1GHz}
- ❑ Also Find Execution time for 350 instructions.

Instruction Category	% Occurrence	Cycles of Instruction
ALU	40	2
Load & Store	15	3
Branch	35	4
Other	10	5

$$f_{clock} = 1 \text{ GHz}$$

$$\rightarrow CPI_{avg} = \frac{\sum_{i=1}^x \eta_i CPI_i}{\sum_{i=1}^x \eta_i} = \frac{40 \times 2 + 15 \times 3 + 35 \times 4 + 10 \times 5}{100} = 3.15$$

$$\rightarrow MIPS = \frac{f_{clock}}{CPI_{avg} \times 10^6} = \frac{10^9}{3.15 \times 10^6} = 317.46 \text{ MIPS}$$

$$\rightarrow t = \frac{n \times CPI_{avg}}{f_{clock}} = \frac{350 \times 3.15}{10^9} = 1102.5 \times 10^{-9} \text{ sec} = 1.1025 \times 10^{-6} \text{ sec} = 1.1025 \text{ } \mu\text{sec}.$$

- **GATE 2014 CS** – Consider two processor P1 and P2 executing the same instruction set. Assume that under identical conditions, for the same input, a program running on P2 takes 25% less time but incurs 20% more CPI as compared to the program running on P1. If the clock frequency of P1 is 1GHz, then the clock frequency of P2 (in GHz) is ?

$$\begin{array}{c} P_1 \\ \hline n_1 = n \\ t_1 = t \\ CPI_1 = CPI \\ f_1 = 1 \text{ GHz} \end{array}$$

$$\begin{array}{c} P_2 \\ \hline n_2 = n \\ t_2 = 0.75t \\ CPI_2 = 1.2 \text{ CPI} \\ f_2 = ? \end{array}$$

$$\rightarrow t = \frac{n \cdot CPI_{avg}}{f_{clock}} \Rightarrow n = \frac{t \cdot f_{clock}}{CPI_{avg}}$$

$$\begin{array}{l} \Rightarrow n_1 = n_2 \\ \Rightarrow \frac{t_1 f_1}{CPI_1} = \frac{t_2 f_2}{CPI_2} \end{array} \quad \left| \quad \begin{array}{l} \Rightarrow \frac{\cancel{t} \times 1 \text{ GHz}}{\cancel{CPI}} = \frac{0.75\cancel{t} \times f_2}{1.2 \cancel{CPI}} \\ \Rightarrow f_2 = \frac{1 \times 1.2}{0.75} = 1.6 \text{ GHz} \end{array} \right.$$

CPU Performance Parameters in COA

→ CPU performance is based on,

1) Execution time t , $P \propto \frac{1}{t}$

2) Throughput.

→ Speed up factor S ,

$$S = \frac{P_x}{P_y} = \frac{1/t_x}{1/t_y} = \frac{t_y}{t_x} \dots \dots \text{Performance of } x \text{ w.r.t. } y.$$

→ If processor 1 takes 1msec to execute one program and processor 2 takes 5 msec to execute same program. Then,

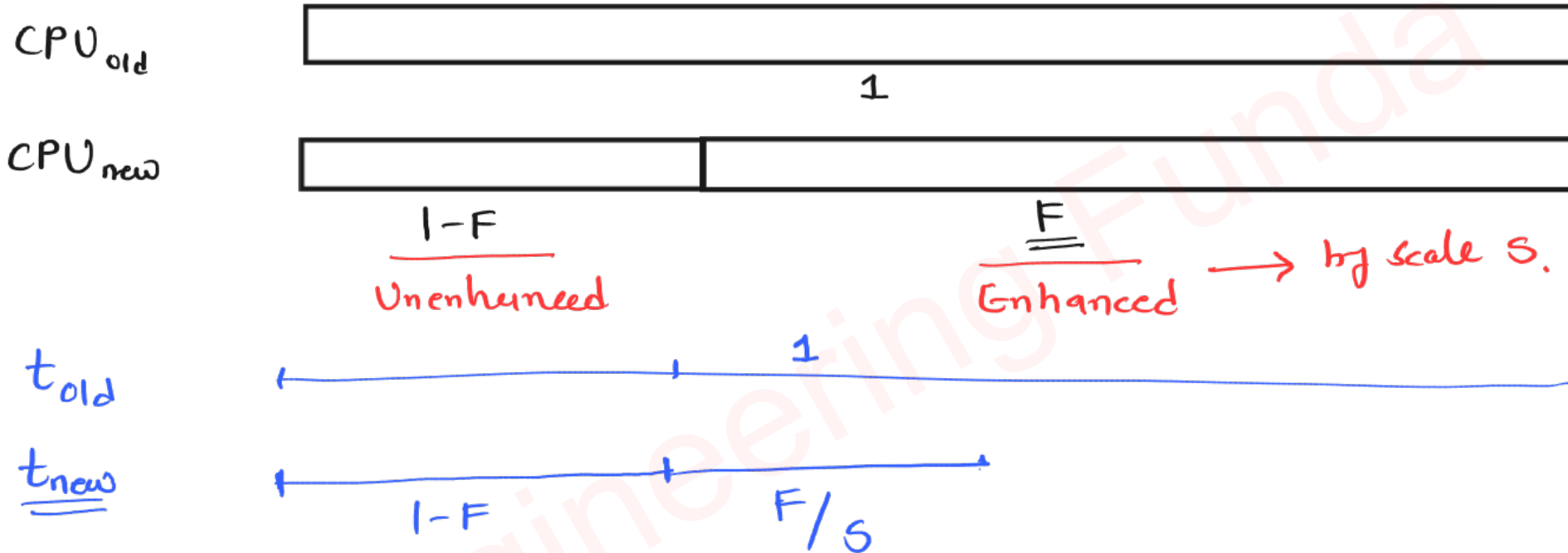
1) Speed up factor of 1 w.r.t. 2

2) Speed up factor of 2 w.r.t. 1.

$$\rightarrow S_{12} = \frac{t_2}{t_1} = \frac{5}{1} = 5 \quad \rightarrow S_{21} = \frac{t_1}{t_2} = \frac{1}{5} = 0.2$$

Amdahl's Law in COA

→ It explains overall speed up factor of given CPU after some enhancement.



$$\rightarrow S_{\text{ave}} = \frac{t_{\text{old}}}{t_{\text{new}}} = \frac{1}{\underbrace{1-F}_{\text{Unenhanced}} + \underbrace{F/S}_{\text{Enhanced}}}$$

❑ **GATE CS** – Assume a microprocessor given which is widely used for scientific applications. It has both integer and floating point instructions. Now floating point instructions are enhanced and made 3 times faster than before and integer instructions are unenhanced. If there are 20% of floating point instructions in the program then find the overall speed up.

$$\rightarrow F = 0.2$$

$$S = 3$$

$$\rightarrow S_{\text{all}} = \frac{1}{(1-F) + \frac{F}{S}}$$

$$= \frac{1}{0.8 + \frac{0.2}{3}}$$

$$= \frac{3}{2.6}$$

$$= 1.1538$$

Examples on CPU Performance in COA

- **GATE CS** – Assume in hypothetical system cache is 20 times faster than main memory and CPU refers to the cache 80% of the time. Calculate the overall speed up of system.

$$\rightarrow F = 0.8$$

$$S = 20$$

$$\rightarrow S_{\text{all}} = \frac{1}{(1-F) + \frac{F}{S}}$$

$$= \frac{1}{0.2 + \frac{0.8}{20}}$$

$$= \frac{20}{4.8}$$

$$= 4.1666$$

❑ **GATE CS** – In hypothetical system, Floating point instructions are improved to run five times as fast, but only 40% of the time was spent on these instructions originally. How much speed up is achieved by new machine?

$$\rightarrow F = 0.4$$

$$S = 5$$

$$\rightarrow S_{\text{all}} = \frac{1}{(1-F) + \frac{F}{S}}$$

$$= \frac{1}{0.6 + \frac{0.4}{5}}$$

$$= \frac{5}{3.4}$$

$$= 1.47$$

❑ **GATE CS** – Assume the frequency of CPU is given as 4.5GHz. Assume we have different type of instructions which have different numbers of instruction count and CPI. Calculate

1. Average instruction execution time

2. MIPS

3. Program execution time

Instruction Type	Instruction Count	CPI
Load	100	10
Store	200	8
Arithmetic	300	5
Logical	150	4
Shift	250	3
Branch	400	2

$$\rightarrow CPI_{avg} = \frac{\sum_{i=1}^x n_i CPI_i}{\sum_{i=1}^x n_i} = \frac{100 \times 10 + 200 \times 8 + 300 \times 5 + 150 \times 4 + 250 \times 3 + 400 \times 2}{100 + 200 + 300 + 150 + 250 + 400}$$

$$\Rightarrow CPI_{avg} = 4.4642$$

$$\rightarrow t_{inst^n} = \frac{CPI_{avg}}{f_{clock}} = \frac{4.4642}{4.5 \times 10^9} = 0.99 \times 10^{-9} \text{ Sec}$$

$$\rightarrow MIPS = \frac{f_{clock}}{CPI_{avg} \times 10^6} = \frac{4.5 \times 10^9}{4.4642 \times 10^6} = 1.008 \times 10^3 \text{ MIPS}$$

$$\begin{aligned} \rightarrow t_{prog} &= n t_{inst^n} \\ &= (100 + 200 + 300 + 150 + 250 + 400) \times 0.99 \times 10^{-9} \\ &= 1386 \times 10^{-9} = 1.386 \mu\text{Sec}. \end{aligned}$$

Examples on CPU Performance in COA

- ❑ **GATE CS** – In a certain processor, we have a program which takes 1 million instructions to execute with processor whose speed is given as 2GHz. Suppose 40% of the instruction takes 2 clock cycles, 30% of the instructions takes 3 clock cycles and remaining 30% takes 5 clock cycles for their respective execution. What is the total execution time for program?

$$\rightarrow n = 10^6$$

$$f_{\text{clock}} = 2 \times 10^9 \text{ Hz}$$

$$\rightarrow \text{CPI}_{\text{avg}} = \frac{\sum_{i=1}^x n_i \text{CPI}_i}{\sum_{i=1}^x n_i} = \frac{0.4 \times 2 + 0.3 \times 3 + 0.3 \times 5}{1} = 3.2$$

$$\rightarrow t_{\text{exe}} = \frac{n \text{CPI}_{\text{avg}}}{f_{\text{clock}}} = \frac{10^6 \times 3.2}{2 \times 10^9} = 1.6 \times 10^{-3} = 1.6 \text{ msec.}$$

- ❑ **GATE CS** – Consider a 2GHz processor which is used to execute a benchmark program with the following instruction and clock cycle count:

Instruction Count	CPI
80000	2
40000	3
5000	3
10000	4

- ❑ What is the effective CPI of the program and total execution time?

$$\rightarrow CPI_{avg} = \frac{\sum_{i=1}^x n_i CPI_i}{\sum_{i=1}^x n_i} = \frac{80000 \times 2 + 40000 \times 3 + 5000 \times 3 + 10000 \times 4}{80000 + 40000 + 5000 + 10000}$$

$$\Rightarrow CPI_{avg} = 2.4814$$

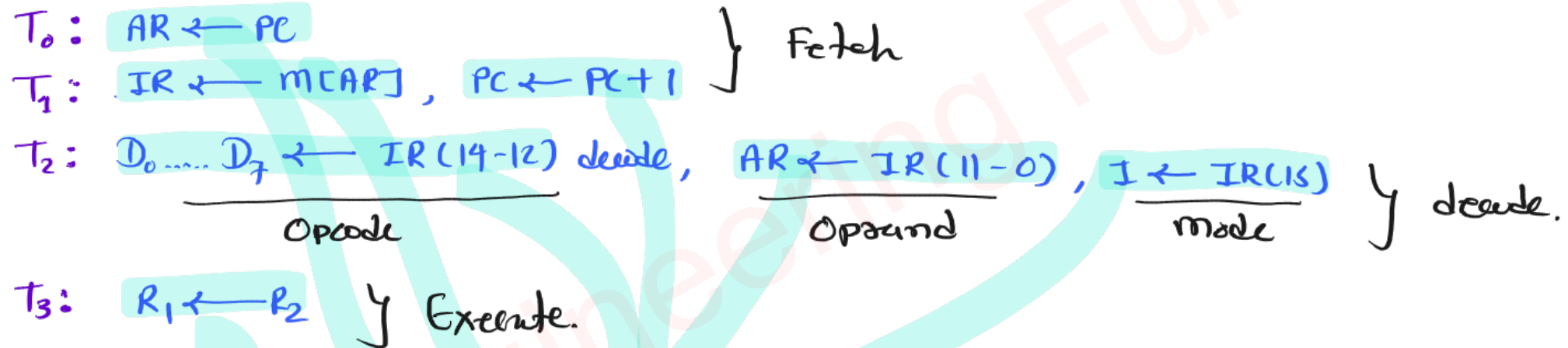
$$\rightarrow t_{exe} = \frac{n \cdot CPI_{avg}}{f_{clock}} = \frac{(80000 + 40000 + 5000 + 10000) \times 2.4814}{2 \times 10^9} = 167.494 \times 10^{-9}$$
$$= 167.494 \times 10^{-6} = 167.494 \mu\text{sec} = 0.167494 \text{ msec.}$$

Micro Operations of Instruction in COA

❖ Basics of Micro-operations

- ❑ There can be multiple Micro Operations in single instruction execution.
- ❑ Micro Operations are executed on values stored in registers.
- ❑ To describe Micro Operations we use "Register Transfer Language - RTL"

Instruction cycle [$\text{mov } R_1, R_2$]



Hardwired Control Unit in COA

❖ Basics of Hardwired Control Unit

- ❑ Control Unit generates control signals to perform microoperations in CPU.
- ❑ Hardwired Control Unit provides control signals by Hardware.
- ❑ In general three types of hardwired control units are available.

1. State Table Method of Hardwired Control Unit

- ❑ It is the most basic method of hardwired control unit.
- ❑ Here behavior of control unit is represented in the form of a table called state table.
- ❑ The rows represents T states and columns represents instructions.

T States	Instructions			
	I1	I2	---	In
T1	C(1, 1)	C(1, 2)	---	C(1, n)
T2	C(2, 1)	C(2, 2)	---	C(2, n)
⋮	⋮	⋮	⋮	⋮
Tm	C(m, 1)	C(m, 2)	---	C(m, n)

❖ Advantage

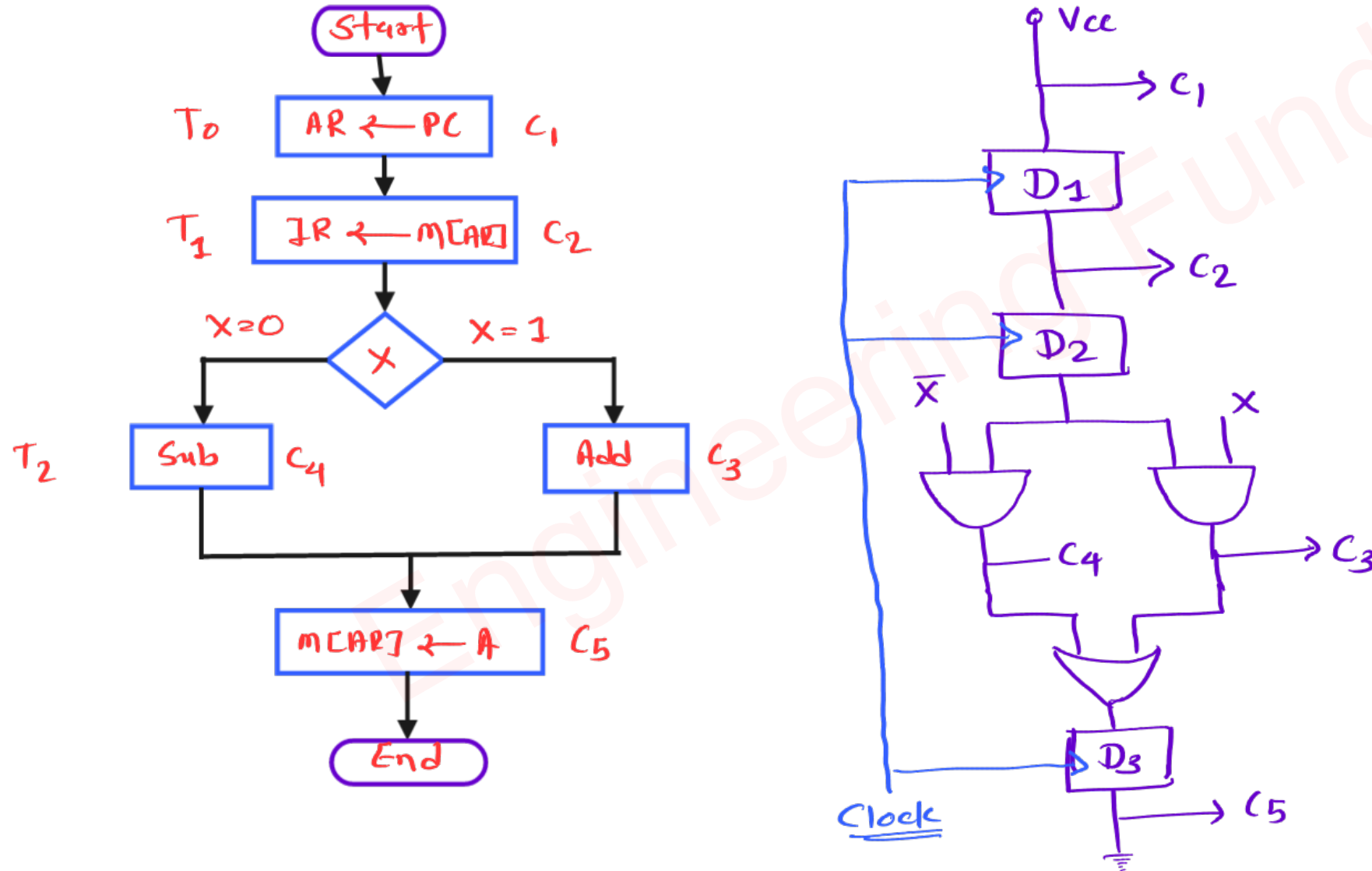
- ❑ It is simple Method.
- ❑ It is ideal for very small instruction set.

❖ Disadvantage

- ❑ For Large instruction set, hardwired circuit will be very complicated and costly.

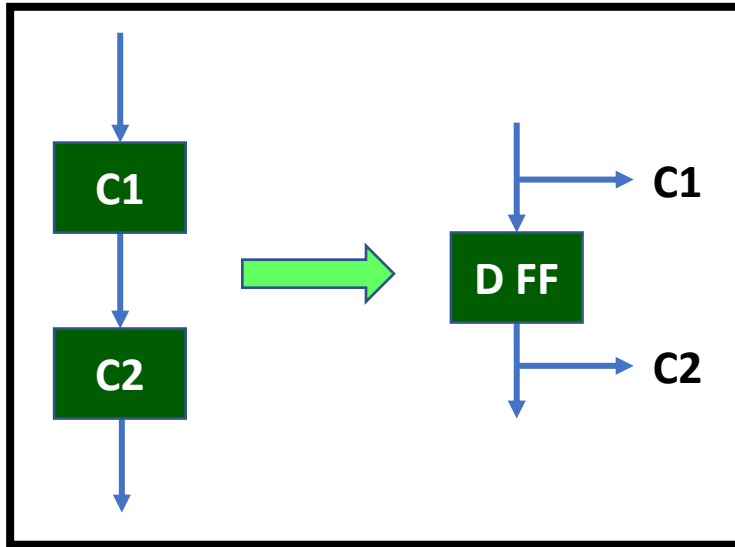
2. Delay Element Method of Hardwired Control Unit

- ❑ Here, behavior of control unit is represented in form of flowchart.
- ❑ Each step of flowchart represents a control signal to be produced.
- ❑ D Flip flops are used to insert delay in-between consecutive control signals.

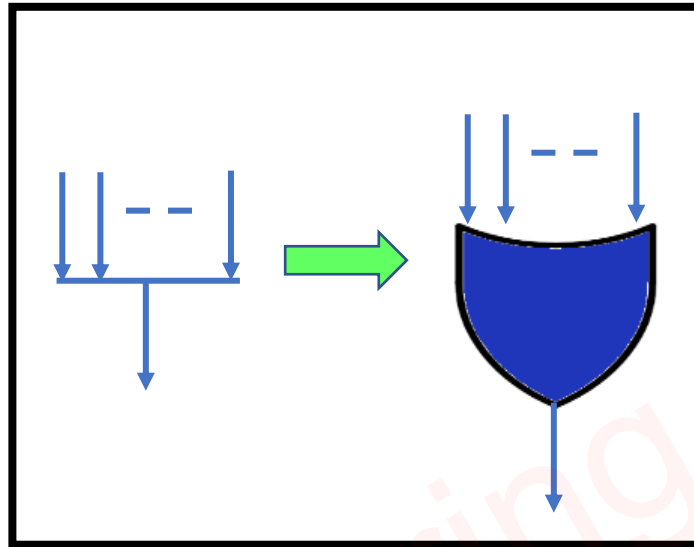


2. Delay Element Method of Hardwired Control Unit

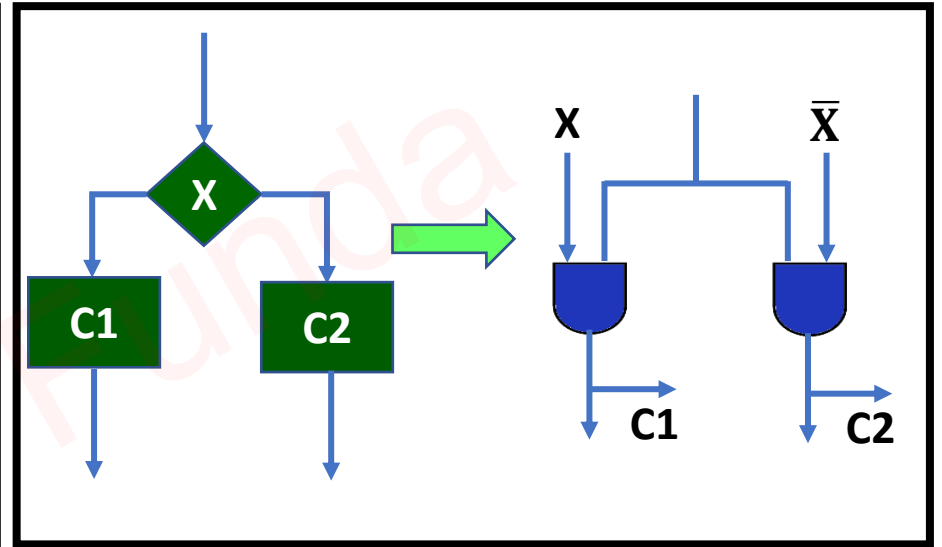
- ❑ Here, Out of all the D Flip Flops only one will be active. So this method is also called “One Hot Method”.
- ❑ We need to apply simple three rules to convert flowchart into hardware control signals.



Rule 1



Rule 2



Rule 3

❖ Advantage

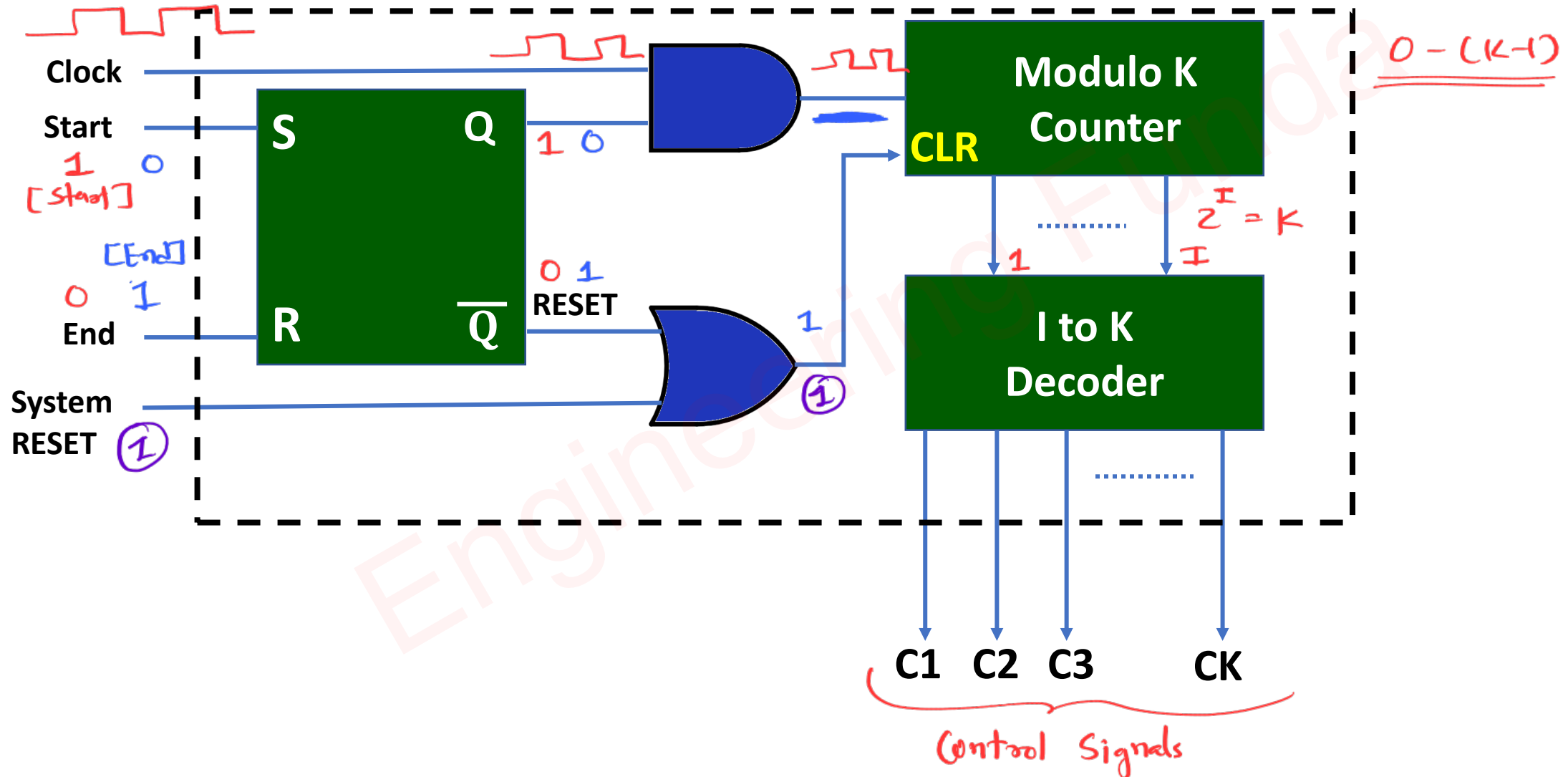
- ❑ It decreases the complexity of circuit for control unit.
- ❑ For Common controls, we don't need separate control signals.

❖ Disadvantage

- ❑ For Large instruction set number of D Flip Flops will be more, which will increase the cost and size of circuit.

3. Sequence Counter Method of Hardwired Control Unit

- ❑ This is most popular method of hardwire control unit.
- ❑ Approach of Delay method and this method is same, but it doesn't require many D Flip Flops for control signals.



3. Sequence Counter Method of Hardwired Control Unit

- ☐ Here, on start signal modulo counter will start counting clock from 0 to (K-1).
- ☐ Based on value of Count, it generates output from 0 to (K-1) using I numbers of terminals.
- ☐ This I numbers of terminals are used for I to K decoder.
- ☐ Based on Count, Decoder output will generate control enable on C1, C2 Ck.
- ☐ At the end of instruction, End signal will reset count to zero.
- ☐ System RESET can also reset count to zero.

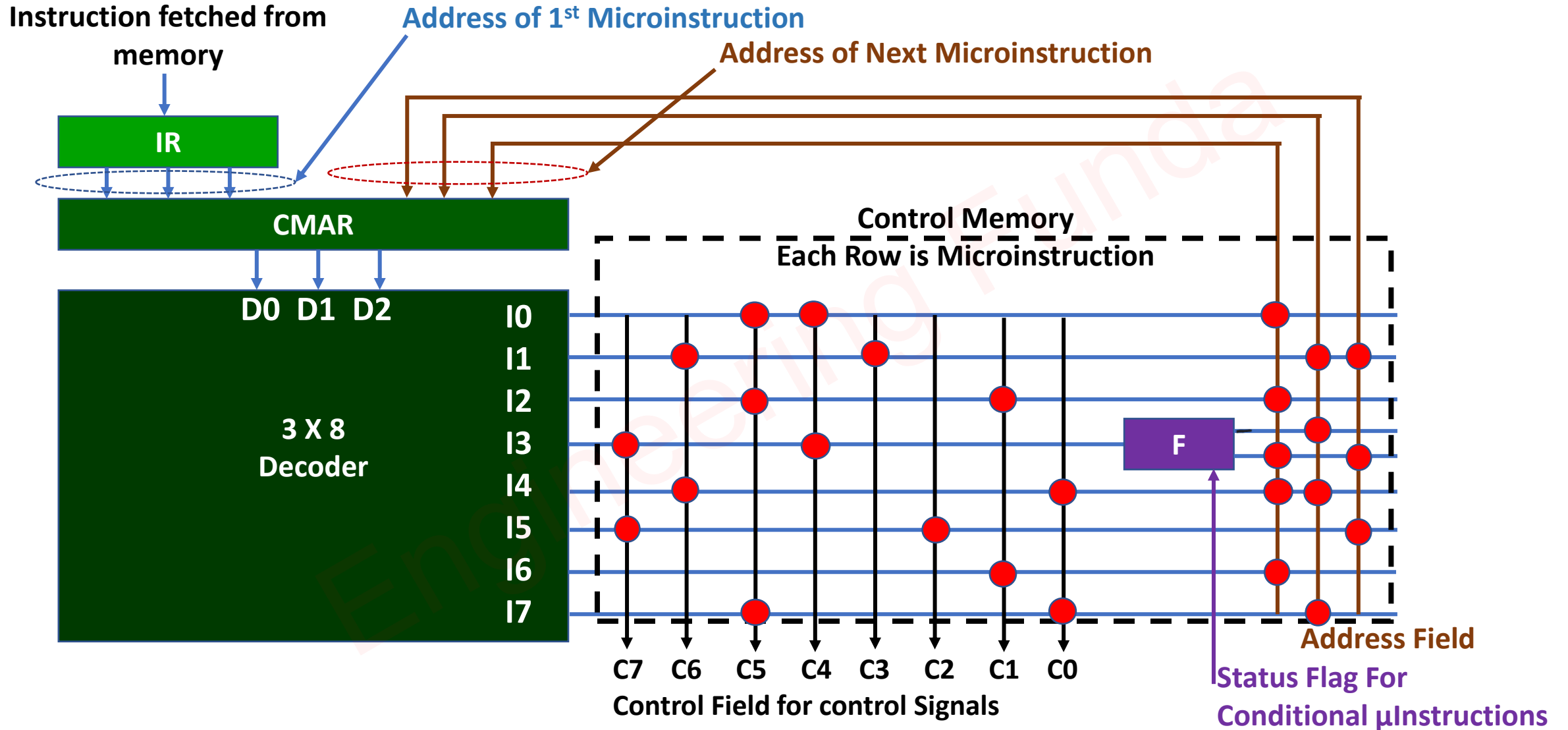
❖ Advantage

- ☐ It avoids too many D Flip Flops in the circuit.

❖ Disadvantage

- ☐ In Modern processor, we have too many instructions support in CPU.
- ☐ So for that hardwire control mechanism is very complex in physical chip.
- ☐ If processor is upgraded then we also needs to redesign the hardwire control again. Which increases the cost.

Wilkes Design For Micro Programmed CU



❖ Wilkes Design for Micro Programmed Control Unit

- ☐ Microprogrammed control unit produces control signals by software, using Microinstructions.
- ☐ A program is set of instructions.
- ☐ Instruction {Microprogram} is set of Microinstructions.
- ☐ Microprograms are stored in control memory, which is there inside processor.
- ☐ 1st when computer fetches the instruction, it goes into IR.
- ☐ Processor uses Opcode of IR, to give 1st address of microinstruction in CMAR.
- ☐ CMAR gives command to decoder for execution of 1st microinstruction.
- ☐ Microinstruction has two fields: Control Field and Address Field
- ☐ Control Field: It generates control signals
- ☐ Address Field: It generates address of next microinstruction.
- ☐ Conditional microinstructions has two address field, based on conditional flag it gives next address of microinstruction.
- ☐ Control Memory is made up of Flash ROM, So it is rewritable and Non volatile.

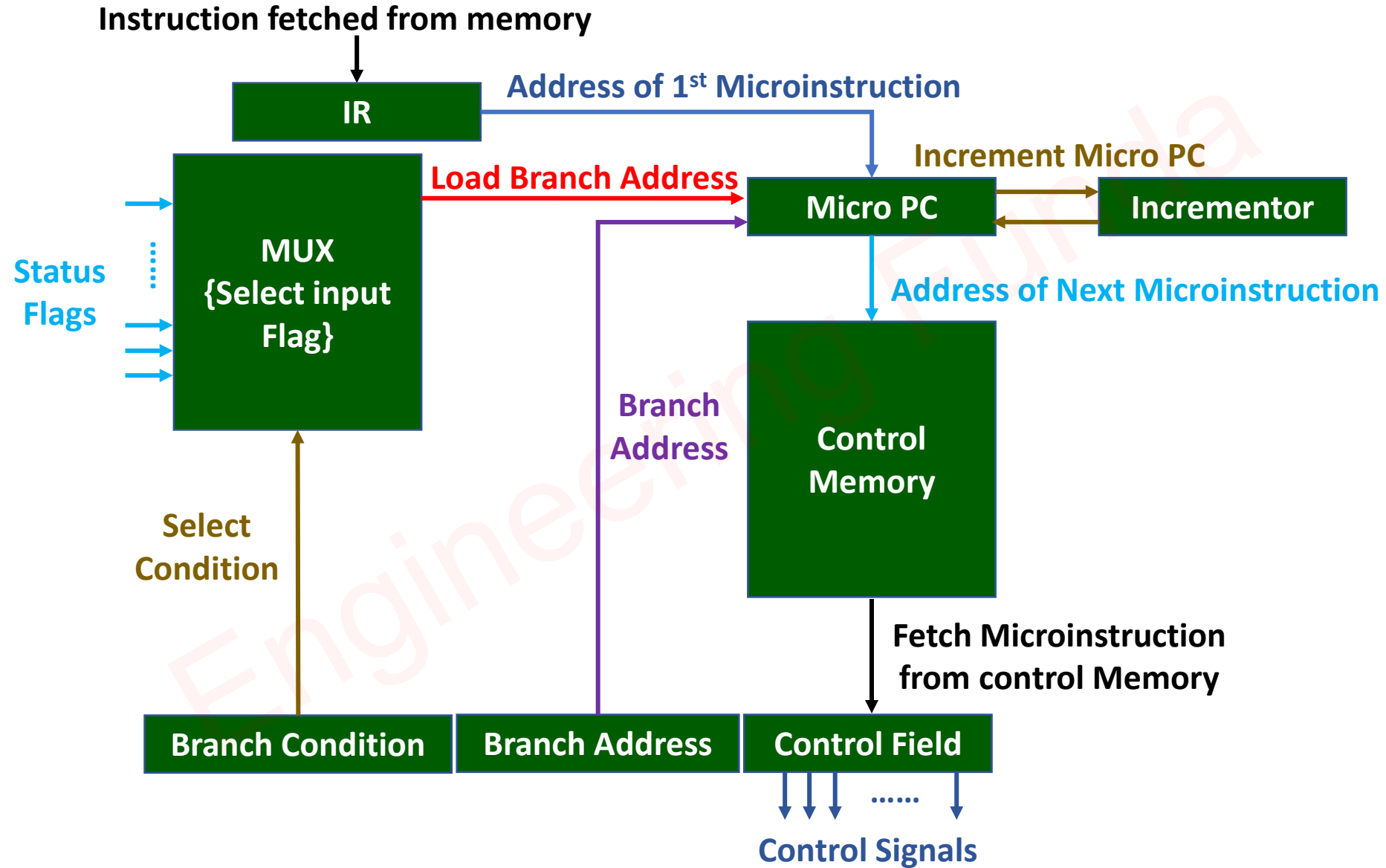
❖ Advantage

- ☐ Flexibility
- ☐ Microinstructions {Microoperations} can be reprogrammed.
- ☐ Debugging is Easy.

❖ Disadvantage

- ☐ Control memory is inside the processor, so it increase the size of processor.
- ☐ So, it also increase the cost of processor.
- ☐ Address field adds more space in control memory.

Basic Micro Programmed Control Unit



❖ Basic Microprogrammed Control Unit

- ☐ Microprogrammed control unit produces control signals by software, using microinstructions.
- ☐ A program is set of instructions.
- ☐ Instruction {Microprogram} is set of microinstructions.
- ☐ Microprograms are stored in control memory, which is there inside processor.
- ☐ 1st when computer fetches the instruction, it goes into IR.
- ☐ Processor uses Opcode of IR, to give 1st address of microinstruction in μ PC.
- ☐ μ PC gives Address of microinstruction to control memory.
- ☐ Microinstruction has three fields: Control Field, branch Address Field and branch condition
- ☐ Control Field: It generates control signals
- ☐ Branch Address Field: It generates address of next microinstruction in case of branch.
- ☐ Branch Condition defines branch with respect to flags condition.
- ☐ MUX has inputs as status flag and selection of inputs is done with respect to branch condition.
- ☐ Control Memory is made up of Flash ROM, So it is rewritable and Non volatile.

❖ Advantage

- ☐ Flexibility
- ☐ Microinstructions {Microoperations} can be reprogrammed.
- ☐ Debugging is Easy.
- ☐ Address field is required only during branch of microinstruction.

❖ Disadvantage

- ☐ Control memory is inside the processor, so it increase the size of processor.
- ☐ So, it also increase the cost of processor.

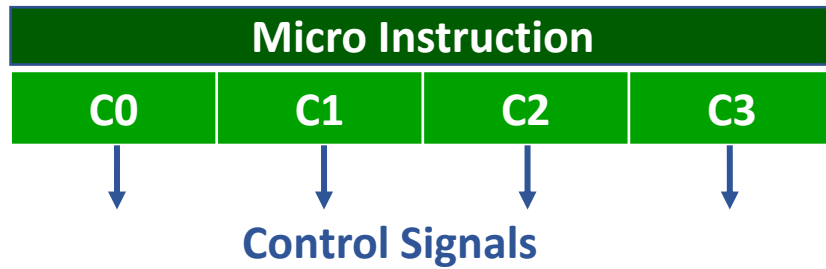
Hardwired CU Vs Microprogrammed CU

Parameters	Hardwired Control Unit	Microprogrammed Control Unit
Control Signals	❖ Generated using Hardware.	❖ Generated using Software {μProgram}
Structure	❖ Based on Hardware, So it's rigid.	❖ Based on Software, So it's flexible.
Modification	❖ Done by redesigning.	❖ Done by reprogramming.
Instruction Set	❖ Small & Simple	❖ Large and Complex
Debugging	❖ Difficult	❖ Easy
Emulation	❖ Not possible	❖ Possible
Execution Speed	❖ Very Fast	❖ Slower
Memory	❖ No memory required	❖ Memory required for μProgram
Cost	❖ Low cost as no control memory, but for many instruction set it is more costly.	❖ High cost due to control memory
Processor	❖ Preferred in RISC Processor	❖ Preferred in CISC Processor
Design Process	❖ Sequential Circuit	❖ Programming
Chip Area	❖ More Chip Area	❖ Less Chip Area
Pipelining	❖ Small and Efficient	❖ Long and Less Efficient
Applications	❖ ARM	❖ INTEL

Horizontal Vs Vertical μ Instruction

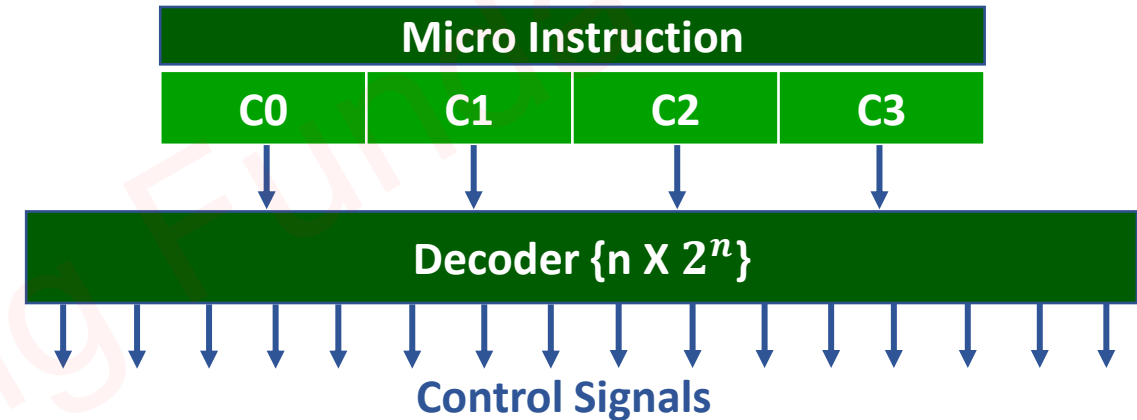
❖ Horizontal Microinstruction

- ❑ Here every bit of microinstruction corresponds to control signal.
- ❑ Based on value of instruction bits directly control signals are generated.



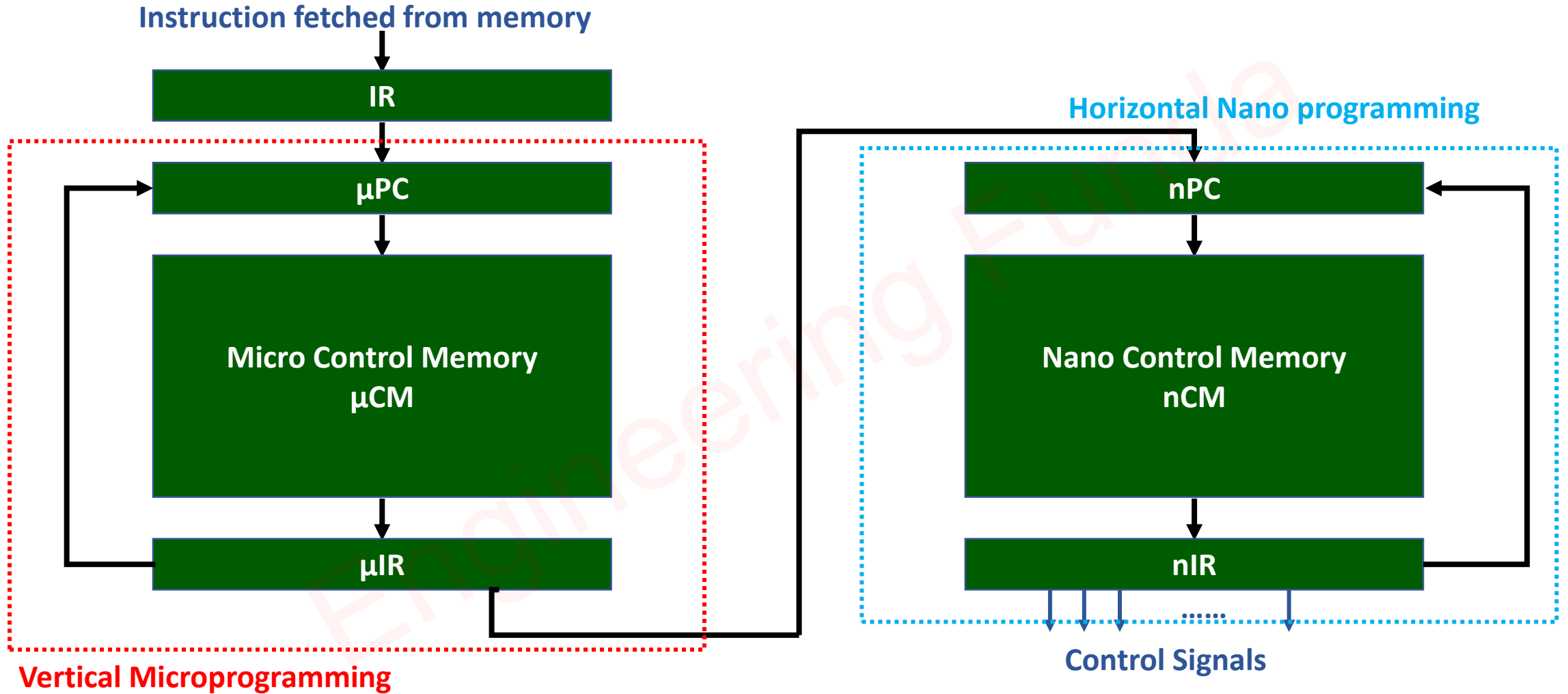
❖ Vertical Microinstruction

- ❑ Here every bit of microinstruction have to be decoded for control signal generation.



Parameters	Horizontal Microinstruction	Vertical Microinstruction
Control Signals	❖ Every bit of microinstruction is control signal	❖ Bits of microinstruction should be decoded 1 st to generate control signals.
Decoder	❖ Not required	❖ Required
Numbers of Signals	❖ Multiple in one microinstruction.	❖ One in one microinstruction
Memory	❖ More memory for more control signals	❖ Less memory
Execution speed	❖ Very Fast	❖ Slower
Control Word	❖ Wide Control word	❖ Short control word

Nano Programming



❖ Nano Programmed Control Unit

- ☐ Horizontal Microprogram control unit are very wide, which requires more control memory space.
- ☐ Vertical Microprogram control unit requires less space but it needs more decoding hardware.
- ☐ Horizontal microprogram control unit has issues regarding memory and vertical microprogram control unit has issues regarding decoding which leads to operating speed.
- ☐ Combination of Horizontal & Vertical microprogram control unit makes nano programming.
- ☐ Here we have two level control memory: 1. μ CM and 2. nCM
- ☐ 1st fetched instruction comes in IR of CPU.
- ☐ Opcode of IR will be given to μ PC, using that we fetch micro instruction into μ PC.
- ☐ Using this we can generate microinstruction from micro control memory, which will be given to μ IR.
- ☐ Section related microoperation is vertical in structure.
- ☐ μ IR address will be given to nPC for nano instruction address.
- ☐ based on nPC, nano control memory generates nano instruction.
- ☐ Nano instruction will be given to nIR, which generates control signals.
- ☐ Section related nano operation is horizontal in structure.

Microinstruction Format in COA

Control field	Address field	flags (F)	Branch Condition (B)
---------------	---------------	-----------	----------------------

↓
Control signals (C)

→ Horizontal
or $\text{Inst.}^n = C \text{ bits}$

→ Vertical
or $\text{Inst.}^n = \log_2 C$

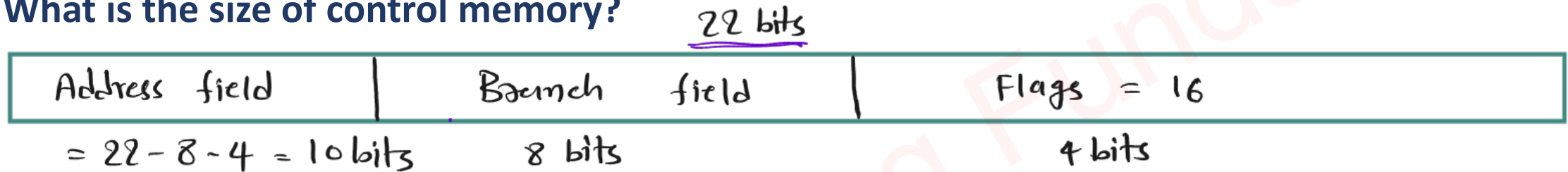
↓
(n)
- No of Control word
- Size of Control memory.
- $2^n \times$ Size of Control word

↓
- No of
bits = $\log_2 F$

↓
- No of
bits = $\log_2 B$

Examples on Microinstruction Format

- ❑ **GATE CS** – Consider a Control Memory Unit in which 22 bits microinstruction is stored.
- ❑ The Microinstruction is having three fields, “Address Field”, “Branch field” has size of 8 bits and the hardware contains 16 flags.
- ❑ What is the size of control memory?



$$\begin{aligned} \rightarrow \text{Control Memory} &= 2^{10} \times 22 \text{ bits} = \frac{2^{10} \times 22}{8} \text{ bytes} \\ &= 22 \text{ Kbits} \\ &= 2816 \text{ bytes.} \end{aligned}$$

❑ **GATE CS** – Consider a Hypothetical Control Unit, which has a total of 233 control signals. How many bits are required in control memory word for horizontal microprogramming and vertical microprogramming?

→ For horizontal microprogramming

- bits = Control signals = 233 bits.

→ For vertical microprogramming.

- bits = $\log_2 C = 8$ bits.

- ❑ **GATE CS** – Consider a microprogrammed control unit design which supports 7 groups of mutually exclusive control signals.

Groups	G1	G2	G3	G4	G5	G6	G7
Control signals	2 (1)	10 (4)	4 (2)	1 (1)	18 (5)	23 (5)	6 (3)

- ❑ How many more control bits are required using horizontal microprogramming over vertical microprogramming?

→ For Horizontal microprogramming = $2 + 10 + 4 + 1 + 18 + 23 + 6 = 64$ bits.

→ For Vertical microprogramming = $1 + 4 + 2 + 1 + 5 + 5 + 3 = 21$ bits.

Ans = $64 - 21 = 43$ bits.

Examples on Microinstruction Format

❑ **GATE CS** – A Consider a hypothetical Control Unit that supports 4K words. The Hardware contains 64 control signals and 16 Flags. What is the size of control word used in bits and control memory in a byte using:

a) Horizontal Programming

b) Vertical programming

Flags = 16 4 bits.	Control signal = 64 H = 64 bits, V = 6 bits.	Addresses = 4K 12 bits.
-----------------------	---	----------------------------

→ Horizontal μ Inst.ⁿ size = $4 + 64 + 12 = 80$ bits.

→ Vertical μ Inst.ⁿ size = $4 + 6 + 12 = 22$ bits.

→ Control memory for Horizontal μ Inst.ⁿ = $2^{12} \times 80$ bits = $\frac{2^{12} \times 80}{8} = 40$ Kbyte.

→ Control memory for vertical μ Inst.ⁿ = $2^{12} \times 22$ bits. = $\frac{2^{12} \times 22}{8} = 11$ Kbyte.

❑ **GATE 2008 IT** – Consider a CPU, where all the instructions require 7 clock cycles to complete execution. There are 140 instructions in the instruction set. It is found that 125 control signals are needed to be generated by the control unit. While designing the horizontal microprogrammed control unit, single address field format is used for branch control logic. What is the minimum size of the control word and control address register?

- a. 125, 7 b. 125, 10 c. 135, 9 ✓ d. 135, 10

Control field = 125 (125 bits)	Address field (10 bits)
--------------------------------	-------------------------

$$\text{No of Address} = 7 \times 140 = 980$$

$$\rightarrow \text{Control word} = 125 + 10 = 135 \text{ bits.}$$

$$\text{Control Address register} = 10 \text{ bits.}$$

Examples on Microinstruction Format

- ❑ **GATE CS** – A Consider a hypothetical Control Unit, which supports 512 address. If the control word is 20 bits long and hardware contains 4 flags and supports 16 branch conditions, what is the maximum number of control signals that can be generated at a time using horizontal microprogramming and vertical microprogramming, respectively?

20 bits			
Control field	Branch Cond. ⁿ = 16	Flags = 4	Address = 512
= $20 - 4 - 2 - 9 = 5$ bits	4 bits.	2 bits	9 bits

- For horizontal microprogramming, control signals = 5
- For vertical microprogramming, control signals = $2^5 = 32$
- Control memory = $2^9 \times 20 = 10240$ bits
= 10 kbits = 1280 bytes.

❑ **GATE CS** – Vertical Microprogramming, Horizontal Microprogramming & Hardwired control are used for control unit design. What is the order of speed of execution from highest to lowest?

1. Hardwired control.
2. Horizontal microprogramming.
3. Vertical microprogramming.

highest
to
lowest.



Examples on Control Unit in COA

- ❑ **GATE 2005 CS** – A hardwire uses 10 control signals S1 to S10, in various time steps T1 to T5, to implement I1 to I4 as shown below:

	T1	T2	T3	T4	T5
I1	S1, S3, S5	S2, S4, S6	S1, S7	S10	S3, S8
I2	S1, S3, S5	S9, S9, S10	S5, S6, S7	S6	S10
I3	S1, S3, S5	S7, S8, S10	S2, S6, S9	S10	S1, S3
I4	S1, S3, S5	S2, S6, S7	S5, S10	S6, S9	S10

- ❑ Find the Boolean expression to generate S5 and S10.

$$\rightarrow S_5 = T_1(I_1 + I_2 + I_3 + I_4) + T_3(I_2 + I_4) = T_1 + T_3(I_2 + I_4)$$

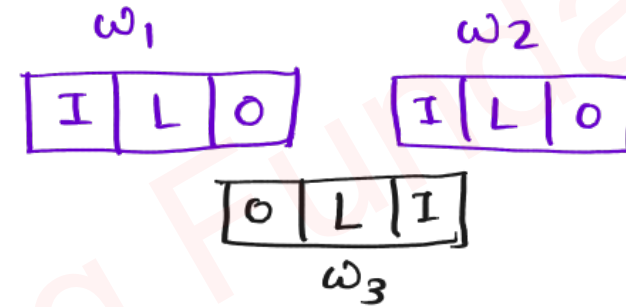
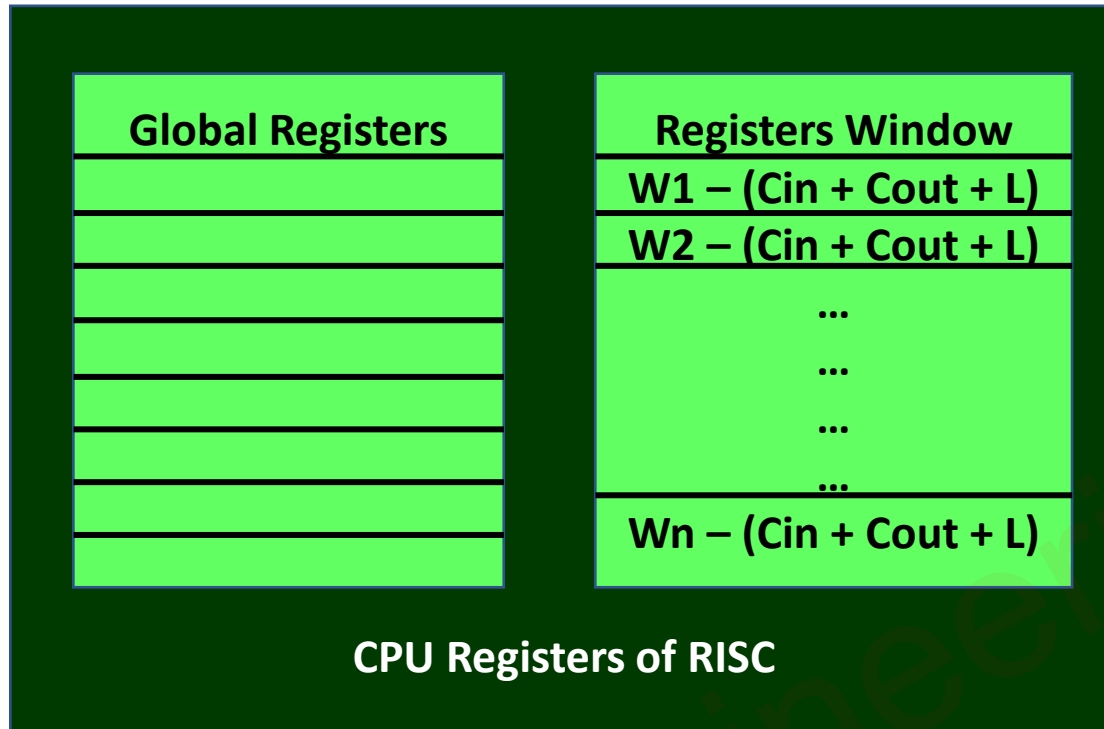
$$\rightarrow S_{10} = T_2(I_2 + I_3) + T_3(I_4) + T_4(I_3) + T_5(I_2 + I_4)$$

- ❑ **GATE CS** – A computer supports five instructions I1, I2, I3, I4 & I5 and four time cycles T1, T2, T3 & T4. Control Signal for computer is $Z = T1 (I1 + I2) + T2 + T3 + T4 (I2 + I3 + I4 + I5)$.
- ❑ During I5 execution, what are the time cycles for execution? *Ans (T2, T3, T4)*

$$Z = T_1 (I_1 + I_2) + T_2 + T_3 + T_4 (I_2 + I_3 + I_4 + I_5)$$

$\frac{T_1}{I_1}$	$\frac{T_2}{I_1}$	$\frac{T_3}{I_1}$	$\frac{T_4}{I_1}$
I_2	I_2	I_2	I_3
	I_3	I_3	I_4
	I_4	I_4	I_5
	I_5	I_5	

Registers Organization in RISC Processor



→ Total Register = G + Reg. of window.

$$T = G + w(L + \max(I, O))$$

❑ **UGCNET CS** – A RISC processor has 700 registers. Each window has 5 input, 8 output & 20 Local registers. If number of windows are 20, then how many global registers are there in RISC processor.

$$\rightarrow T = 700$$

$$I = 5$$

$$O = 8$$

$$L = 20$$

$$\left. \begin{array}{l} I = 5 \\ O = 8 \\ L = 20 \end{array} \right\} \omega = 20$$

$$\Rightarrow T = G + \omega(L + \max(I, O))$$

$$\Rightarrow 700 = G + 20(20 + 8)$$

$$\Rightarrow G = 700 - 20 \times 28$$

$$\Rightarrow \boxed{G = 140}$$